

Writing Assembly Code Basics



where to write a assembly code,,, I don't know anything

No worries! You're just starting — so let's go **step by step**.

✓ Step 1: Choose your platform

Where do you want to write and run Assembly code?

👉 If you're using Windows:

- Use **DOSBox + TASM** (good for old-style 16-bit assembly)
- Or use **Visual Studio** with MASM
- Or use **NASM** with Command Prompt

👉 If you're using Linux (Ubuntu or any distro):

- Use **NASM (Netwide Assembler)** — most popular and modern
- You can write code using **any text editor** (like `gedit`, `nano`, or `nvim`)

✓ Step 2: Install an assembler (like NASM)

For Linux (Ubuntu):

Open terminal and run:

```
bash

sudo apt update
sudo apt install nasm
```

For Windows:

1. Download NASM from <https://www.nasm.us>
2. Add NASM to your system path
3. Use Notepad++ or any editor to write `.asm` files

✓ Step 3: Write your Assembly Code

You can use any code editor:

- On **Linux**:

```
bash

nano hello.asm
```

or

```
bash

nvim hello.asm
```

- Example **hello.asm** (Linux, 64-bit NASM):

```
asm

section .data
    msg db "Hello, World!", 0xA
    len equ $ - msg

section .text
    global _start

_start:
    mov rax, 1          ; syscall: write
    mov rdi, 1          ; stdout
    mov rsi, msg        ; message to print
    mov rdx, len        ; message length
    syscall

    mov rax, 60         ; syscall: exit
    xor rdi, rdi        ; exit code 0
    syscall
```

✅ Step 4: Assemble and Run

For Linux:

```
bash

nasm -f elf64 hello.asm      # Assemble
ld -o hello hello.o         # Link
./hello                     # Run
```

For Windows (DOSBox + TASM):

Use TASM to assemble and run **.asm** files inside DOSBox.

🧠 Tips for Beginners

- Assembly language is very low-level — it's **specific to CPU architecture** (x86, x86_64, ARM)
- NASM is best for beginners on modern systems
- Start with **simple programs** like:

- Print something
- Add two numbers
- Loops and conditionals

? Want me to guide you setting it up on your OS?

Just tell me:

- Are you using **Windows or Linux**?
- 32-bit or 64-bit?
- Do you want to use **TASM**, **NASM**, or something else?

Then I'll guide you 100% like a beginner-friendly tutorial.



I am a linux user,,, Ubuntu 24.04 and 64 bit operating system

Great! You're using **Ubuntu 24.04 (64-bit)** — that's perfect for writing and running **modern Assembly code** using **NASM**.

✓ Step-by-Step Guide to Write and Run Assembly on Ubuntu 64-bit

🔧 Step 1: Install NASM

Open your terminal and run:

```
bash

sudo apt update
sudo apt install nasm
```

✓ This installs the NASM assembler.

📝 Step 2: Write Your First Assembly Code

Use any editor you like. For example, using **nano**:

```
bash

nano hello.asm
```

Paste this **simple "Hello, World!"** program (64-bit Linux NASM syntax):

```
asm

section .data
    msg db "Hello, World!", 0xA    ; Message with newline
    len equ $ - msg               ; Length of the message

section .text
    global _start

_start:
    mov rax, 1                    ; syscall: write
    mov rdi, 1                    ; file descriptor: stdout
    mov rsi, msg                  ; address of the string
    mov rdx, len                  ; length of the string
    syscall

    mov rax, 60                   ; syscall: exit
    xor rdi, rdi                  ; exit code 0
    syscall
```

Save and exit:

- Press **Ctrl+O** → Enter to save
- Press **Ctrl+X** to exit

Step 3: Assemble and Link

Run these commands in the terminal:

```
bash

nasm -f elf64 hello.asm    # Assemble into object file
ld -o hello hello.o        # Link into executable
```

✓ You will get a file named **hello**

Step 4: Run the Executable

```
bash

./hello
```

 Output:

```
Hello, World!
```

✓ **You're Done!**

You just wrote, assembled, linked, and ran your first 64-bit Assembly program on Ubuntu 🎉

Try Another: Add Two Numbers

Create a file:

```
bash  
  
nano add.asm
```

Paste:

```
asm  
  
section .text  
    global _start  
  
_start:  
    mov rax, 5  
    add rax, 3      ; rax = 5 + 3  
  
    ; exit with result as return code  
    mov rdi, rax  
    mov rax, 60  
    syscall
```

Assemble, link, and run:

```
bash  
  
nasm -f elf64 add.asm  
ld -o add add.o  
./add  
echo $?
```

Output:

```
8
```

Want to do input/output, loops, or system calls next?

Just tell me what type of program you want — I'll guide you step-by-step.